

2026 US LLVM Developers' Meeting

27 October 2026

How to build a Python DSL for MLIR

Extracting DkgDSL's reusable core into mlir dsl

Guray Ozen

Principal Compiler Engineer, NVIDIA

Amir Tavakkoli

Why a Python DSL for MLIR?



Test a dialect

Public → Internal → LLVM



Reuse building blocks

JIT · Cache · Environment



Generate recipe IR

Python → Recipe IR → Consumer

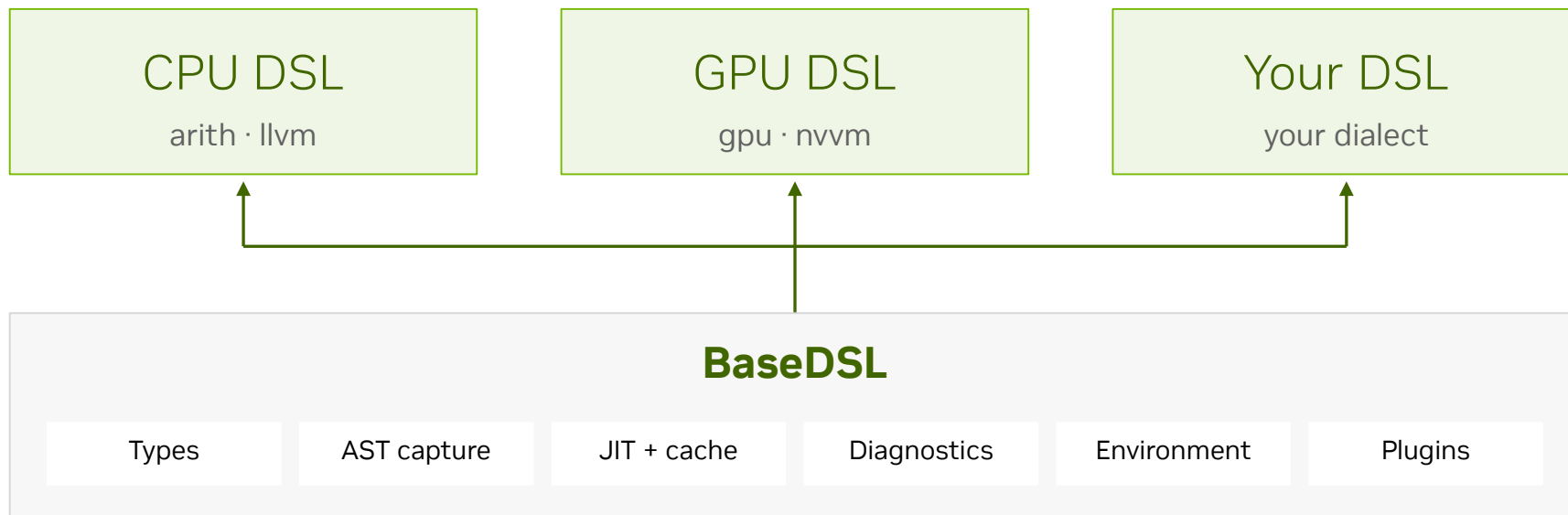


Build a language

NVIDIA CuTe DSL / CUTLASS Python

[NVIDIA documentation](#)

One core. Many DSLs.



Design principles

01 No new language.

Objects · imports · libraries

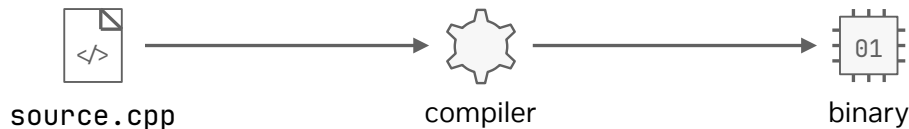
02 Python stays Python.

MLIR through a DSL namespace

This leads to **multi-stage programming**.

What is multi-stage programming?

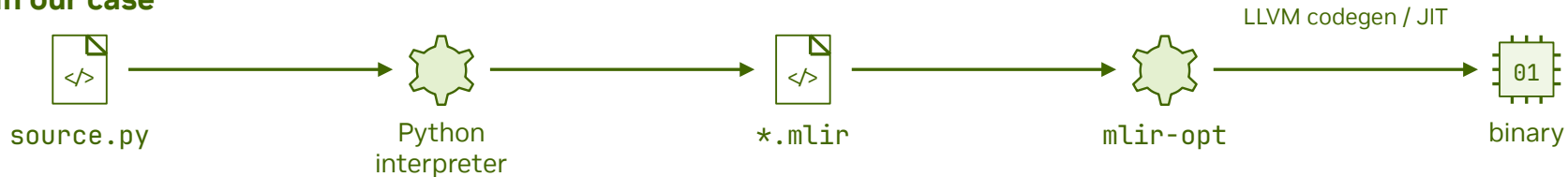
Traditional compiler



Multi-stage compilers



In our case



Why multi-stage programming?

01 METAPROGRAM

All of Python.

Objects · imports · libraries

02 DSL NAMESPACE

Explicit capture.

Types · operators · builders

Inside `@m.jit · import mlir.dsl as m`

`range(4)`



Python loop

Runs now

`m.range(4)`



scf.for

Runs later

How to build a Python DSL for MLIR

Python

Python

```
import mlir.dsl as m

@m.jit
def scale(x: m.Int32, factor):
    return x * factor

scale(4, 3)
```

Objects · imports · metaprogramming

Trace: execute Python

Meta

factor = 3
Run Python

Staged

x = SSA
Emit MLIR

MLIR

Generated MLIR

```
// x is a runtime i32
%c3 = arith.constant 3 : i32
%r = arith.muli %x, %c3 : i32
llvm.return %r : i32
```

Compile

Test

Consume IR

One program. Two binding times.

Python / Meta

RUNS NOW

Objects · functions · dictionaries

DSL / Staged



compile

RUNS LATER

Typed values · MLIR operations

Agenda

- 01 BaseDSL + plugins
- 02 Types + metaprogramming
- 03 Compilation + control flow
- 04 Amir: mutation tracking
- 05 Tile IR: a sub-DSL

Start with BaseDSL

Sub-DSL setup

```
class MySubDSL(m.BaseDSL):  
    plugins = [MyDialectPlugin()]  
  
    def __init__(self):  
        super().__init__(  
            name="MY_DSL",  
            dsl_package_name=["mydsl"],  
            **target_config)
```

Inherit

Capture · compile · execute

Customize

Dialect · passes · runtime

BaseDSL building blocks

Service	Interface
Name	DSL identity
Environment	MY_DSL_*
Plugins	Dialects · libraries · hooks
Pipeline	Passes + options
JIT	Trace → compile → execute
Name mangling	Specialized symbols

Environment, per DSL

Environment manager

```
class MyEnv(EnvironmentVarManager):  
    fast_math: bool = env_var(  
        "FAST_MATH", default=False,  
        affects_compile=True)  
  
class MySubDSL(m.BaseDSL):  
    _env_class = MyEnv  
    # name="MY_DSL" in its constructor
```

Namespace

```
MY_DSL_PRINT_IR  
MY_DSL_CACHE_DIR
```

Cache identity

```
affects_compile=True
```

Plug in your dialect

Downstream plugin · implementation excerpt

```
class MyDialectPlugin(m.DialectPlugin):  
    name = "mydialect"  
    def register_dialects(self, context):  
        mydialect.register_dialect(context)  
    def pipeline_passes(self):  
        return ["convert-mydialect-to-llvm"]  
  
class MySubDSL(m.MlirDSL):  
    plugins = [MyDialectPlugin()]
```

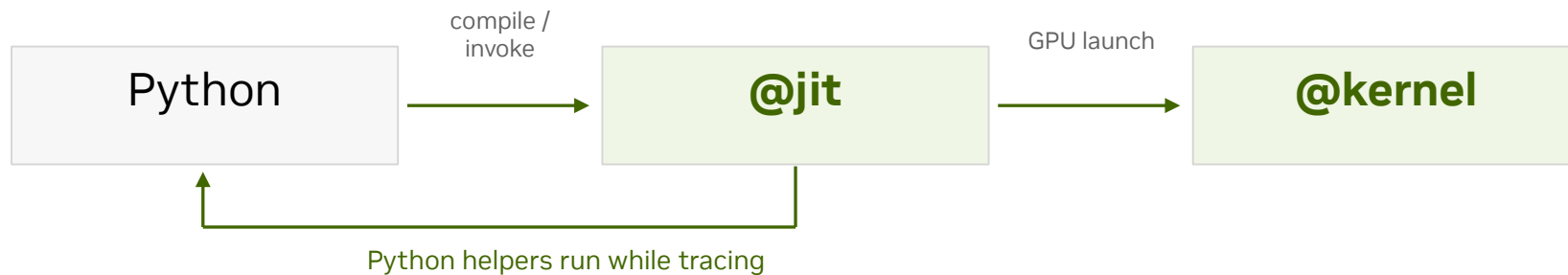
Register

Dialect bindings

Lower

Passes + runtime libraries

@jit and @kernel



Host

Specialize · compile · call

GPU

Build kernel + launch

Direct launch

Python → @kernel
Target policy

Types: Python values → MLIR

Bool · i1

Int / UInt · 8–64

FP64/32/16 · BF16

FP8/6/4 · TF32

Python or SSA value $\longrightarrow$ dtype + promotion $\longrightarrow$ arith operations

Conversion

Python

MLIR

Capture

`m.Int32(7) → staged use`

`arith.constant : i32`

Promote / upcast

`x32 + y64`

`extsi → addi : i64`

Downcast

`y64.to(m.Int32)`

`arith.trunci`

Int → float

`x32.to(m.Float32)`

`arith.sitofp`

Float → int

`f32.to(m.Int32)`

`arith.fptosi`

Error

`x32.to(int)`

`Runtime ≠ Python value`

Beyond scalars

Value	Building blocks
Numeric	Promotion · cast · dtype · align
Pointer	Load / store · GEP · address space
Struct / frozen record	Immutable fields
Vector	Splat · sum · lanes
Interop + math	Torch dtypes · math helpers

Extend with `mydialect.Tensor`.

Local type inference

Annotations + local inference

```
@m.jit
def affine(x: m.Int32, factor):
    return x * factor + 1

affine(4, 3)

# x          → runtime i32
# factor     → known Python int
# result     → i32
```

Boundary

Annotation → runtime type

Operator

Promote → emit → wrap

Join

Same structure + types

Your arithmetic, your dialect

Value wrapper · sketch

```
class MyNumber:
    def __init__(self, value):
        self.value = value
    def __add__(self, rhs):
        op = myarith.AddOp(
            self.value, rhs.value)
        return MyNumber(op.result)
```

Dialect plugin · sketch

```
class MyArithPlugin(m.DialectPlugin):
    def register_dialects(self, context):
        myarith.register_dialect(context)
    def pipeline_passes(self):
        return ["convert-myarith-to-llvm"]
```

Add mydialect.Tensor

Register a Tensor leaf

```
class Tensor:
    def __init__(self, value):
        self.value = value

m.register_leaf(
    Tensor,
    ir_types=lambda t: [t.value.type],
    ir_values=lambda t: [t.value],
    from_ir_values=lambda old, values:
        Tensor(values[0]))
```

Value protocol

Flatten / rebuild

Runtime boundary

Argument adapter

Python values specialize the program

Python

```
@m.jit
def affine(x: m.Int32, scale, bias):
    return x * scale + bias

affine(7, 3, 4) # 25
```

Generated MLIR

```
%c3 = arith.constant 3 : i32
%c4 = arith.constant 4 : i32
%a = arith.muli %x, %c3 : i32
%r = arith.addi %a, %c4 : i32
llvm.return %r : i32
```

scale, bias: Meta · x: Staged

Objects + dictionaries → arithmetic

Python polymorphism

```
class Op:
    def apply(self, x):
        raise NotImplementedError

class AddN(Op):
    def __init__(self, n): self.n = n
    def apply(self, x): return x + self.n

class MulN(Op):
    def __init__(self, k): self.k = k
    def apply(self, x): return x * self.k
```

Metaprogram

```
@m.jit
def pipeline(x: m.Int32, ops: m.Constexpr):
    for op in ops.values():
        x = op.apply(x)
    return x

ops = {"shift": AddN(2), "scale": MulN(3)}
pipeline(4, ops) # 18

# Captured: (x + 2) * 3
```

No runtime dictionary or virtual dispatch.

The same recipe, on a GPU

Array GPU interface · extraction target

```
@m.kernel
def fused(arr: m.Array,
          ops: m.Constexpr):
    tx, _, _ = m.thread_idx()
    x = arr[tx]
    for op in ops.values():
        x = op.apply(x)
    arr[tx] = x

# In @m.jit:
# fused(arr, ops).launch(grid=..., block=...)
```

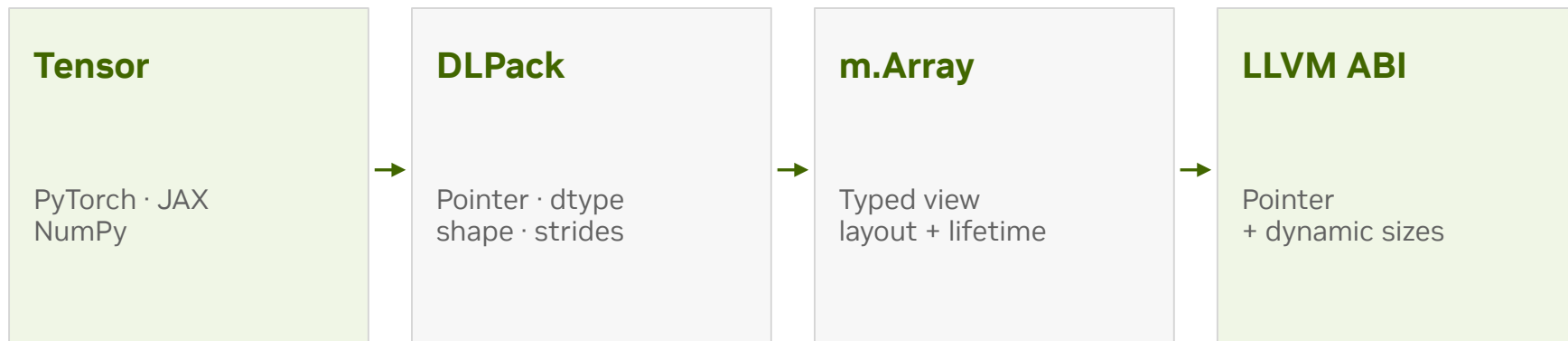
Meta

Objects · dispatch · recipe loop

Staged

Thread ID · load · add · store

Framework tensors → m.Array



DkgDSL capability → mlir dsl extraction target

Compile. Run. Export.

Compile API · extraction target

```
compiled = m.compile[m.GenerateLineInfo(True)](  
    launch, a_sig, b_sig, out_sig)  
  
compiled(a, b, out)  
compiled.export_to_c("build", "vector_add")  
  
# IR for a recipe consumer:  
artifact = m.compile.to_precompiled_mlir(  
    recipe, *signature)
```

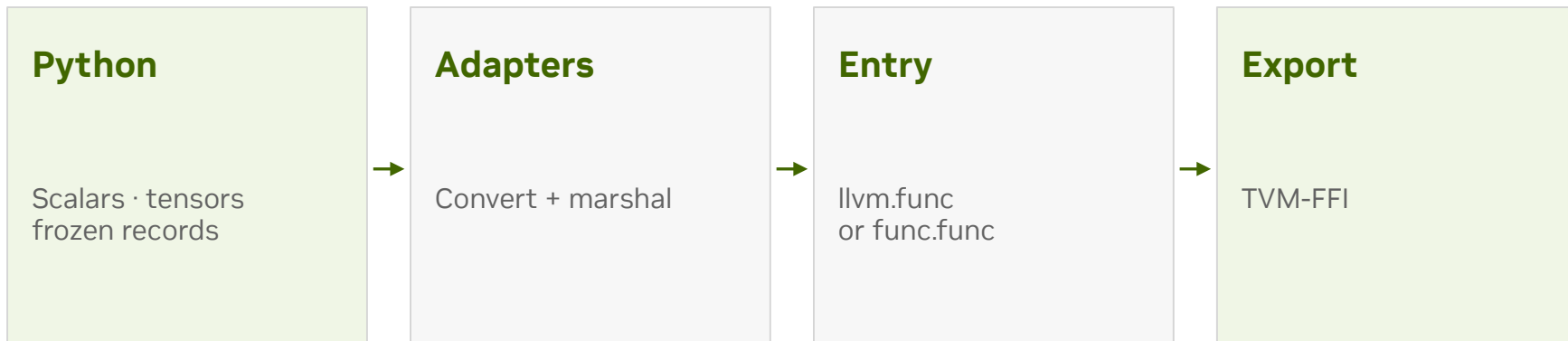
Compile

Signature + options

Use the artifact

Execute · C export · MLIR

Adapters + export plugins



Optional: `MLIR_DSL_ENABLE_TVM_FFI=1`

Errors + MLIR remarks

Source error + fix

```
acc = 0
for i in range(n):
    acc += i

# error[PHASE_MUTATE_PYTHON]
# `acc` is a Python value.
# Create a runtime value:
# acc = m.Int32(0)
```

MLIR_DSL_DEBUGINFO=1 · MLIR_DSL_REMARKS=.*

Errors

File · line · fix

Remarks

Trace + passes → records

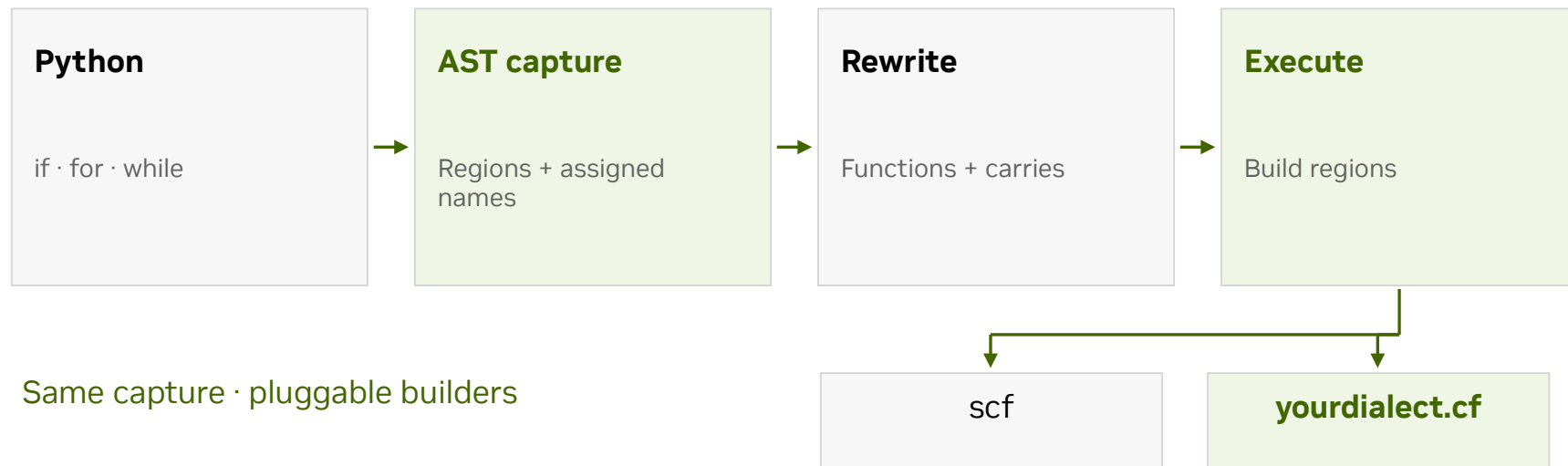
See what the compiler does

Tool	Setting / output
Source locations	Python line → MLIR location
Verify trace	VERIFY_TRACE
Remarks	REMARKS · REMARKS_OUTPUT
Inspect IR	PRINT_IR · KEEP_IR
Profile	PROFILE_COMPILER

Cache compilation work

Cache	Stores	Hit
Memory	Callable + engine	Reuse callable
Disk	Lowered bytecode	Skip passes
Identity	IR + settings + toolchain	Compatible programs

Control flow needs the AST



Operators capture expressions. The AST captures structure.

Assigned name → iter_arg

Python · find the carry

```
acc = m.Int32(0)
for i in range(n):
    acc = acc + i

# Defined outside: acc
# Written inside:  acc
# write_args = ["acc"]
```

MLIR · thread the SSA value

```
%sum = scf.for %i = %c0 to %n
      step %c1 iter_args(%acc = %zero)
      → (i32) : i32 {
    %next = arith.addi %acc, %i : i32
    scf.yield %next : i32
  }
```

Flatten → build region → yield → rebuild

Frozen records carry staged state

Immutable record

```
from dataclasses import dataclass, replace

@dataclass(frozen=True)
class State:
    total: m.Int32
```

Rebind across the loop

```
@m.jit
def sum_record(n: m.Int32):
    state = State(m.Int32(0))
    for i in range(n):
        state = replace(state,
                        total=state.total + i)
    return state.total

sum_record(10)  # 45
```

New SSA values → a new record

Swap the control-flow dialect

Control-flow plugin · sketch

```
class MyControlFlow(m.DialectPlugin):
    def install(self, dsl):
        super().install(dsl)
        dsl.executor.set_functions(
            **mydialect_executors)

class MySubDSL(m.BaseDSL):
    plugins = [MyControlFlow()]
    # Configure BaseDSL as before.
```

Default

scf.for · if · while

Your dialect

Custom regions + yields

Or build regions explicitly

Explicit control flow

```
@m.jit(preprocess=False)
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i, current, result in m.for_(
        0, n, 1, [acc]):
        m.yield_([current + i])
    return result
```

Builders

for_ · if_ · while_ · yield_

Boolean helpers

and_ · or_ · not_
any_ · all_

Python → MLIR → 45

Python · executed

```
@m.jit
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i in range(n):
        acc += i
    return acc

sum_upto(10)  # 45
```

Generated MLIR

```
%sum = scf.for %i = %c0 to %n
      step %c1 iter_args(%a = %zero)
      → (i32) : i32 {
    %next = arith.addi %a, %i : i32
    scf.yield %next : i32
  }
llvm.return %sum : i32
```

Fuse a Python recipe

Python · NumPy buffer

```
@m.jit
def fused(arr: m.Pointer[m.Float32],
         ops: m.Constexpr):
    for i in m.range(4):
        x = arr[i]
        for op in ops.values():
            x = op.apply(x)
        arr[i] = x

# [0, 1, 2, 3] → [6, 9, 12, 15]
```

Loop body · MLIR

```
%p = llvm.getelementptr %arr[%i]
      : (!llvm.ptr, i32) → !llvm.ptr, f32
%x = llvm.load %p : !llvm.ptr → f32
%a = arith.addf %x, %two : f32
%r = arith.mulf %a, %three : f32
llvm.store %r, %p : f32, !llvm.ptr
scf.yield
```

Staged array loop · Meta recipe loop

Richer mutation

Objects · aliasing · staged control flow

— Guray Ozen

Advanced mutation tracking

PyIR · Amir Tavakkoli

— Amir Tavakkoli

Tile IR: reuse BaseDSL

Tile IR backend · excerpt

```
class CudaTileIR(BaseDSL):
    _env_class = CudaTileIREnvironmentManager
    def __init__(self):
        super().__init__(
            name="CUDA_TILE_DSL",
            dsl_package_name=["tile_ir"],
            compiler_provider=compiler_provider,
            pass_sm_arch_name="compute-capability",
            device_compilation_only=True,
            preprocess=True)
```

Shared core

Capture · types · diagnostics

Target policy

Tile IR · codegen · launch

Tile IR: supply the loop builder

Tile IR loop builder · excerpt

```
def build_loop():
    def body(iv, *carried):
        updated = captured_body(iv, *carried)
        tile.loop_continue(tuple(updated))

    return tile.for_loop(
        body, lower_bound=start,
        upper_bound=stop, step=step,
        init_values=list(write_args))
```

Shared

Body + carried values

Tile IR

for + continue

Tile IR: a typed Python kernel

mlir dsl interface · target sketch

```
import tile_ir.cuda_tile_dsl as tile

@tile.kernel
def sum_upto(n: tile.Int32):
    acc = tile.constant(0)
    for i in range(n):
        acc += i
    tile.print_tko("sum=%i\n", [acc])
```

Tile IR · loop body

```
%sum = for %i in (%zero to %n,
    step %one) : tile<i32>
    iter_values(%acc = %zero)
    → (tile<i32>) {
    %next = addi %acc, %i : tile<i32>
    continue %next : tile<i32>
}
```

Tile IR: compile for the GPU

Backend routes · excerpt

```
# Offline path: Tile IR bytecode → cubin
writeBytecode(output, module.operation)
cubin = self._tileiras_compile_cubin(path)

# Alternative: CUDA driver JIT
library = cuda_helpers.load_library_from_file(path)
kernel = cuda_helpers.get_library_kernel(
    library, name)
```

Offline

Bytecode → tileiras → cubin

Driver JIT

Bytecode → CUDA driver

Tile IR: call the kernel

CUDA Python launch · excerpt

```
module = cuda_helpers.load_cubin_module(cubin)
kernel = cuda_helpers.get_kernel_function(
    module, name)
cuda_helpers.launch_kernel(
    kernel, num_tiles, (1, 1, 1), stream,
    smem_size=0, kernel_args=packed_args)
```

Python call

```
sum_upto(10,
        num_tiles=[1, 1, 1])
```

Expected

```
sum=45
```

Shared core. Your dialect, compiler, runtime.